

COURSE REGISTRATION SYSTEM

1. Functional Requirements

The Course Registration System supports three types of users: Students, Lecturers, and Administrators.

- Students can log in to the system, view available courses, register for courses, cancel course registrations, and view their registered courses.
- Administrators can manage student information, lecturer information, courses, and course sections.
- Lecturers can view the list of students enrolled in the courses they are assigned to teach.

2. Non-Functional Requirements

- The system must ensure data security and user privacy.
- User passwords must be encrypted.
- The system should respond to user requests within 3 seconds.
- The system must support multiple users accessing the system at the same time.
- The system should be reliable and available during registration periods.

3. Data Flow Diagram

- DFD Level 0 shows the overall interaction between users (Student, Lecturer, Admin) and the Course Registration System.
- DFD Level 1 breaks the system into main processes including authentication, course management, course registration, and reporting.

4. Use Case Diagram

- The main actors are Student, Lecturer, and Admin.
- Use cases include login, course registration, course cancellation, course management, and viewing enrollment reports.

5. Class Diagram

The main classes in the system include:

- User
- Student
- Lecturer
- Course
- CourseSection
- Registration

These classes represent the core entities and their relationships in the system.

6. Data Model

The database design includes the following tables:

- STUDENT
- LECTURER
- COURSE
- COURSE_SECTION
- REGISTRATION

Each table has a primary key, and foreign keys are used to represent relationships between students, courses, and course sections.

7. Interface Design

The system uses a console-based interface with numbered menu options to allow users to navigate the system easily based on their roles.

8. Implementation

The system is implemented using Python and follows object-oriented programming principles such as encapsulation and modularity.

9. Testing (Polished)

Testing is performed using test cases to verify:

- User login
- Course registration and cancellation
- Detection of schedule conflicts
- Credit limit validation